

Phase 3 du projet

MTH8211

Oussama Mouhtal, Yasmine Amami

Description du projet

Assimilation de données

Ce projet traite le problème d'**assimilation de données** : ajuster l'état initial d'un modèle dynamique à partir d'observations bruitées et d'une information *a priori*.

Formulation 4D-Var

La fonction coût 4D-Var s'écrit

$$\min_{x_0 \in \mathbb{R}^n} J(x_0) = \|x_0 - x_b\|_{B^{-1}}^2 + \sum_{t_i \in \mathcal{T}} \|y_i - \mathcal{H}_i(\mathcal{M}_{t_0, t_i}(x_0))\|_{R_i^{-1}}^2, \quad (1)$$

où $\mathcal{M}_{t_0, t_i}$ propage l'état du temps t_0 à t_i , et $\mathcal{H}_i$ est l'opérateur d'observation au temps t_i . Les matrices $B \in \mathbb{R}^{n \times n}$ et $R_i \in \mathbb{R}^{m_i \times m_i}$ sont symétriques définies positives (covariances d'erreur de l'information à priori et des observation).

Modèle Lorenz 95

Le **modèle de Lorenz-95** est un système dynamique simplifié souvent utilisé comme modèle mathématique jouet pour étudier certaines propriétés des systèmes chaotiques et des phénomènes en météorologie.

Le modèle de Lorenz-95 décrit un ensemble de n variables continues, généralement notées $x_1, x_2, \dots, x_n$. Chacune de ces variables représente une grandeur qui évolue dans le temps en fonction de ses interactions avec les autres variables. La dynamique des variables dans le modèle de Lorenz-95 est gouvernée par les équations du type :

$$\frac{dx_i}{dt} = -x_{i-2}x_{i-1} + x_{i-1}x_{i+1} - x_i + F, \quad 1 \leq i \leq n$$

où l'on impose des *conditions aux limites périodiques*, à savoir : $x_{-1} = x_{n-1}$, $x_0 = x_n$ et $x_{n+1} = x_1$. La constante F est un paramètre du modèle, généralement fixé à $F = 8$.

- Le modèle sera représenté par un objet `Lorenz95Model`.
- Le système d'équations 195 est intégré en utilisant un schéma Runge Kutta explicite, le modèle de propagation $\mathcal{M}_{t_0, t_i}$ obtenu est appelée `traj`. Le code est présent dans l'annexe.

Opérateur d'observation

L'opérateur d'observation $\mathcal{H}_i : \mathbb{R}^n \rightarrow \mathbb{R}^{m_i}$ extrait ou transforme des composantes de l'état au temps t_i . Nous utilisons un opérateur d'observation linéaire qui sélectionne les indices dans l'espace où les observations sont

présent. L'opérateur d'observation sera modéliser par un objet `ObsOperator` (Pour plus de description voir l'annexe). La matrice de covariance d'observation est diagonale et $\sigma_i = \sigma_R$.

$$R_i = \sigma_R^2 I_{m_i}.$$

Génération des observations

Les observations y_i proviennent d'une vérité x^{true} :

$$y_i = \mathcal{H}_i(\mathcal{M}_{t_0, t_i}(x^{\text{true}})) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, R_i).$$

Information *a priori* (background)

Le background x_b est une perturbation gaussienne de la vérité x^{true} :

$$x_b = x^{\text{true}} + \eta, \quad \eta \sim \mathcal{N}(0, B), \quad B = \sigma_B^2 I_n.$$

Problématique (révisée)

Résolution des sous problèmes d'assimilation données avec les méthodes itératifs de la bibliothèque `Krylov`.

Test des opérateur tangentielle et leur adjoint

```
n = 40
nt = 10
dt = 0.01
F = 8.0
model = Lorenz95Model(F, dt)      # Modèle de Lorenz95

# Pour les tests de l'observation
sigmaR = 0.5
total_space_obs = 10
total_time_obs = 5
space_inds_obs = round(Int, range(1, n; length=total_space_obs))
time_inds_obs = round(Int, range(1, nt-1; length=total_time_obs))
m = total_space_obs*total_time_obs
obs = ObsOperator(sigmaR, space_inds_obs, n, time_inds_obs, nt, m,
model)      # L'opérateur d'observation H

@testset "TLM-ADJ (Model : M)" begin
    x = randn(n)
    dx = randn(n)
    a = randn(n)

    y1 = tlm_traj(model, x, dx, nt)      # Le jacobien du modèle au point x : J_M(x)
    y2 = ad_traj(model, x, a, nt)        # Le transposé du jacobien au point x : J_M(x)^T

    inner1 = dot(y1, a)                  # < J_M(x) dx, a >
    inner2 = dot(dx, y2)                  # < dx, J_M(x)^T a >

    @test inner1 inner2 atol=1e-7         # < J_M(x) dx, a > = < dx, J_M(x)^T a >
end

@testset "TLM vs Finite Differences (Model : M)" begin
```

```

x = randn(n)
dx = randn(n)
    = 1e-6

fx = traj(model, x , nt)
fx_perturbed = traj(model, x .+ .* dx, nt)
fd_approx = (fx_perturbed .- fx) ./

tlm_val = tlm_traj(model, x, dx, nt)

@test fd_approx tlm_val atol=10*
end

@testset "TLM-ADJ Consistency (Opérateur d'observation H)" begin
    x = randn(n)
    dx = randn(n)
    ay = randn(length(space_inds_obs))

    y1 = tlm_hop(obs, dx)          # J_H(x) = J_H (L'opérateur d'observation H est linéaire)
    y2 = adj_hop(obs, ay)          # J_H^T

    inner1 = dot(y1, ay)           # < J_H*dx, ay >
    inner2 = dot(dx, y2)           # < dx, J_H^T*ay >

    @test inner1 inner2 atol=1e-7 # < J_H*dx, ay > = < dx, J_H^T*ay >
end

@testset "TLM vs Finite Differences (Opérateur d'observation : H)" begin
    x = randn(n)
    dx = randn(n)
        = 1e-6

    fx = hop(obs, x)
    fx_perturbed = hop(obs, x .+ .* dx)
    fd_approx = (fx_perturbed .- fx) ./

    tlm_val = tlm_hop(obs, dx)

    @test fd_approx tlm_val atol=10*
end

@testset "TLM vs Finite Differences (G = H o M Composition du model et Obs)" begin
    x = randn(n)
    dx = randn(n)
        = 1e-6

    fx = gop(obs, x)
    fx_perturbed = gop(obs, x .+ .* dx)
    fd_approx = (fx_perturbed .- fx) ./

    tlm_val = tlm_gop(obs, x, dx)

```

```

    @test fd_approx tlm_val atol=10*
end

@testset "gop: TLM-ADJ Consistency (G operator)" begin
    xt = randn(n)
    dx = randn(length(xt))
    a = randn(length(space_inds_obs) * length(time_inds_obs))

    y1 = tlm_gop(obs, xt, dx)
    y2 = adj_gop(obs, xt, a)

    inner1 = dot(y1, a)
    inner2 = dot(dx, y2)

    @test inner1 inner2 atol=1e-7
end

```

```

Test Summary:          | Pass  Total  Time
TLM-ADJ (Model : M) |    1      1  0.6s
Test Summary:          | Pass  Total  Time
TLM vs Finite Differences (Model : M) |    1      1  0.0s
Test Summary:          | Pass  Total  Time
TLM-ADJ Consistency (Opérateur d'observation H) |    1      1  0.0s
Test Summary:          | Pass  Total  Time
TLM vs Finite Differences (Opérateur d'observation : H) |    1      1  0.0s
Test Summary:          | Pass  Total  Time
TLM vs Finite Differences (G = H o M Composition du model et Obs) |    1      1  0.0s
Test Summary:          | Pass  Total  Time
gop: TLM-ADJ Consistency (G operator) |    1      1  0.1s

```

```
Test.DefaultTestSet("gop: TLM-ADJ Consistency (G operator)", Any[], 1, false, false, true, 1.7739201957)
```

Code 4Dvar

- A chaque utilisation d'un **sous-solveur**, on affiche les informations suivantes (**verbose** = 1) :
 - Le **nombre d'itérations** nécessaires pour converger
 - Le **résidu initial** ($\|r_0\|$)
 - Le **résidu final** ($\|r_f\|$)
 - Le temps d'exécution
 Ces données sont reportées à intervalles réguliers pendant les itérations de **Gauss-Newton**.

Remarque : les critères d'arrêt utilisés pour les sous-solveurs sont ceux définis par défaut.

- Enfin, un tableau récapitulatif présente les statistiques globales pour chaque sous-solveur (**verbose** = 2) :
 - Le **nombre total d'itérations Gauss-Newton** effectuées pour satisfaire le critère d'arrêt
 - Le nombre totale d'itérations des sous solveurs → La **norme du gradient** final
 - La **valeur de la fonction coût non linéaire** finale
 - L'**erreur relative** par rapport à la vérité (norme relative de l'erreur)
 - Le temps totale d'exécution des sous solveurs
- Les critères d'arrêt de la boucle externe sont définis par défaut comme suit :

$$\text{max_outer} = 25, \quad \text{atol_grad} = 10^{-10}, \quad \text{rtol_grad} = 10^{-10}$$

```

function nonlinear_funcval(x, y, obs, xb, B, R)
    eo = y .- gop(obs, x)
    eb = x .- xb
    return eb'* invdot(B, eb) + eo'*invdot(R, eo)
end

function quadratic_funcval(x, dx, y, obs, xb, B, R)
    eo = gop(obs, x) .- y .+ tlm_gop(obs, x, dx)
    eb = x .- xb .+ dx
    return dot(eb, invdot(B, eb)) + dot(eo, invdot(R, eo))
end;

"""
    fourdvar(model, obs, R, B, xt, xb, y;
            max_outer=25, atol_grad=1e-10, rtol_grad=1e-10,
            prec=false, k=50, p=10, =5, verbose=0)

Réalise une assimilation de données 4D-Var incrémentale sur le modèle choisi
(p.ex. Lorenz-95) en comparant plusieurs sous-solveurs de Krylov pour le
sous-problème quadratique de Gauss-Newton.

# Args
- `model` : objet modélisant le modèle dynamique (ex. `Lorenz95Model`).
- `obs` : opérateur d'observation.
- `R` : matrice de covariance d'observation.
- `B` : matrice de covariance de background.
- `xt` : « vérité » (état de référence).
- `xb` : background (a priori) - point de départ de Gauss-Newton.
- `y` : vecteur des observations.

# Keys-Args
- `max_outer` : nombre max d'itérations externes de Gauss-Newton.
- `atol_grad` : tolérance absolue sur la norme du gradient.
- `rtol_grad` : tolérance relative sur la norme du gradient.
- `prec` : active le préconditionnement (catégorie NE uniquement).
- `k` : nbre de valeur propre dominant à sélectionner
- `p` : sur-échantillonnage pour revd.
- `` : période de mise à jour du préconditionneur (tous les `` outer iters; aussi à l'itération 1).
- `verbose` : 1 = affiche les stats des sous-solveurs à quelques itérations GN ;
              2 = affiche également le tableau récapitulatif global.

# Sous-solveurs testés
- **NE** (équations normales) : `minres_qlp`, `minres`, `cg_lanczos`, `cg`, `minares`, `diom`, `dqgmres`
- **LS** (moindres carrés) : `lsqr`, `lsmr`.

Quand `prec=true`, un préconditionneur `P` est construit par REVD (`revd`) sur la Hessienne
de Gauss-Newton et réutilisé/rafraîchi toutes les `` itérations externes.
Les méthodes `lsqr`, `lsmr`, `minares`, `diom`, `dqgmres` sont ignorées si `prec=true`.
"""

function fourdvar(model, obs, R, B, xt, xb, y;
    max_outer = 25,
    atol_grad = 1e-10,
    rtol_grad = 1e-10,

```

```

prec = false,
k = 50,
p = 10,
  = 5,
q = 0,
verbose = 0)

n = length(xb)
xa = copy(xb)

iterates_GN = Dict{String, Vector{Float64}}{ }()

# Les sous solveurs utiliser
methods = Dict(
  "NE" => ["minres_qlp", "minres", "cg_lanczos", "cg", "minares", "diom", "dqgmres"],
  "LS" => ["lsqr", "lsmr"]
)

row_names = vcat("Outer-Iter", "Inners-iter", "||grad||", "||f(x_k)||", "||err||", "time")
header = [""]
global stats_GN_table = row_names

for (category, algos) in methods
  for name in algos
    if prec && (name in ["lsqr", "lsmr", "minares", "diom", "dqgmres"])
      continue
    end
    push!(header, name)
    # Initialisation des itérées de Gauss-Newton
    iterates_GN[name] = copy(xa)
    nb_iter = 0
    b = zeros(n)
    normb = normb0 = 0.0
    (verbose == 1) && println("Résolution su problème avec le sous-solveur $name")
    global stats_subsolver = vcat("Inner-Iter", "||r0||", "||rf||", "time")
    header_subsolver = [""]
    time = 0
    P = I
    inner_outer = 0
    for iter_outer in 1:max_outer

      d = misfit(obs, y, iterates_GN[name])
      b = invdot(B, xb .- iterates_GN[name]) .+ adj_gop(obs, iterates_GN[name], invdot(R, d))
      if category == "LS"
        rhs = [sqrtinvdot(R, d) ; sqrtinvdot(B, xb .- iterates_GN[name])]
        A = Jacobian4DVar(obs, R, B, iterates_GN[name])
      elseif category == "NE"
        rhs = b
        A = Hessian4DVar(obs, R, B, iterates_GN[name])
        if prec && (iter_outer% == 0 || iter_outer == 1)
          S, Λ, , _ = revd(A, k; p=p, q=q)
          P = Prec(S, Λ, )
        end
      end
    end
  end
end

```

```

        end
    end

    if iter_outer == 1
        normb = normb0 = norm(b)
    else
        normb = norm(b)
    end

    solver = getproperty(Krylov, Symbol(name))
    if name == "diom"
        dx, stats = solver(A, rhs; memory=50, M = P, history=true)
    else
        dx, stats = solver(A, rhs; M = P, history=true)
    end
    iterates_GN[name] .+= dx
    time += stats.timer
    nb_iter += 1
    inner_outer += stats.niter
    if normb < atol_grad + rtol_grad*normb0
        break
    end
    if iter_outer%5 == 0 || iter_outer == 1
        push!(header_subsolver, "GN iter $iter_outer")
        global stats_subsolver = hcat(stats_subsolver, vcat(stats.niter, stats.residuals[1],
stats.residuals[end], stats.timer))
    end

end

(verbose == 1) && pretty_table(stats_subsolver, header=header_subsolver)

res_final = norm(b)
fval = nonlinear_funcval(iterates_GN[name], y, obs, xb, B, R)
err_xt = norm(xt - iterates_GN[name]) / norm(xt)
global stats_GN_table = hcat(stats_GN_table, vcat(nb_iter, inner_outer, res_final, fval, err_xt))
end
end
if verbose == 2 && prec == false
    println("Statistiques GN (partie 1)")
    ncols = size(stats_GN_table, 2) # nombre total de colonnes
    mid = ceil(Int, ncols/2) # point de coupure

    header1 = header[1:mid]
    header2 = header[mid+1:end]

    pretty_table(stats_GN_table[:, 1:mid], header=header1)

    println("\nStatistiques GN (partie 2)")
end

```

```

        pretty_table(stats_GN_table[:, mid+1:end], header=header2)
    elseif verbose == 2 && prec == true
        println("statistiques sur GN en utilisons différents sous-solveurs")

        pretty_table(stats_GN_table, header=header)
    end

    return iterates_GN
end;

```

Test du code 4dvar

```

n = 60
nt = 6
dt = 0.025
F = 8.0
sigmaR = 0.1
total_space_obs = 10
total_time_obs = 3      # Le nombre d'observation m = total_space_obs * total_time_obs
sigmaB = 0.8

# Model
model = Lorenz95Model(F, dt)

# Observations

space_inds_obs = round.(Int, range(1, n; length=total_space_obs))
time_inds_obs = round.(Int, range(1, nt-1; length=total_time_obs))
m = total_space_obs*total_time_obs
obs = ObsOperator(sigmaR, space_inds_obs, n, time_inds_obs, nt, m, model)
R = RMatrix(sigmaR)

# Background
B = BMatrix(sigmaB, n)

# Spin-up : amener le modèle sur son régime
xt = 3.0 .* ones(n) .+ randn(n) # état initial arbitraire
xt = traj(model, xt, 5000)      # amener le modèle sur son régime (l'état vérité)

# Background state
xb = xt .+ randn(n) .* sigmaB

# Observations
y = generate_obs(obs, xt)

max_outer = 20

@testset "FourDVar : amélioration vs background & baisse du coût" begin
    iterates_GN =
        fourdvar(model, obs, R, B, xt, xb, y;

```



```

max_outer=max_outer, prec=false)

# coh rence
@test haskey(iterates_GN, "minres_qlp")
@test haskey(iterates_GN, "minres")
@test haskey(iterates_GN, "cg")

# 1) La solution doit am liorer le background xb
err_bg = norm(xt - xb) / norm(xt)
for (name, xa) in iterates_GN
    err_an = norm(xt - xa) / norm(xt)
    @test err_an <= err_bg + 1e-8
end

# 2) Le co t non lin aire J(xa) doit baisser par rapport   J(xb)
J_bg = nonlinear_funcval(xb, y, obs, xb, B, R)
for xa in values(iterates_GN)
    @test nonlinear_funcval(xa, y, obs, xb, B, R) <= J_bg + 1e-10
end
end;

```

Test Summary:		Pass	Total	Time
FourDVar : am�lioration vs background & baisse du co�t		21	21	2.3s

Pr conditionnement

Pour acc l rer les sous-solveurs de Krylov (cat gorie NE) dans 4D-Var, on emploie un *pr conditionneur spectral* construit   partir des k -paires propres dominantes (S_k, Λ_k) de la (Gauss-)Hessienne   l'it ration j :

$$A_j = B^{-1} + G_j^\top R^{-1} G_j.$$

On d finit

$$P = I + S_k \left(\sqrt{\theta} \Lambda_k^{-\frac{1}{2}} - I_k \right) S_k^\top,$$

avec θ choisie comme la plus petite valeur propre utiliser pour construire le pr conditionneur (λ_k si les valeurs sont ordonn es d'une fa on d croissante en fonction de leur index).

Quand employer ce pr conditionneur ?

Ce pr conditionneur est particuli rement pertinent lorsque la matrice est un op rateur, pr sente une **forte d croissance spectrale** et/ou un ** cart spectral**. Dans ce contexte, l'estimation des paires propres dominantes par des m thodes al atoires **revd** est   la fois **efficace** et **peu co teuse**.“

Utilisation pratique en assimilation 4D-Var.

- Estimer (S_k, Λ_k) par randomized evd (eigenvalue decomposition) sur A_j puis construire $P = \text{Prec}(S, \Lambda, \theta)$ (voir Annexe).
- Rafra chir p riodiquement le pr conditionneur (tous les ℓ it r s de Gauss-Newton/ le choix de ℓ est heuristique).
- Choisir k pour un bon compromis co t/acc l ration.

Test de l'op rateur Prec

```

@testset "Prec : op rateur de pr conditionnement (Float64)" begin
    using Random, LinearAlgebra, Test
    Random.seed!(2025)

```

```

n = 40
k = 5
    = collect(range(20.0, 2.0; length=n))
S = Matrix{Float64}(I, n, n)[: , 1:k]
Λ = [1:k]
    = minimum(Λ)
P = Prec(S, Λ, )

# 1) Formule explicite vs mul!
x = randn(n)
y_op = zeros(Float64, n)
mul!(y_op, P, x)

d = (sqrt()./sqrt.(Λ) .- 1.0)
P_exp = I + S * Diagonal(d) * S'
y_exp = P_exp * x
@test isapprox(y_op, y_exp; atol=1e-12, rtol=0)

# 2) Action sur la base propre : P*s_j = (√/√_j) s_j
for j in 1:k
    sj = S[:, j]
    yj = zeros(Float64, n)
    mul!(yj, P, sj)
    @test isapprox(yj, (sqrt()/sqrt(Λ[j])) * sj; atol=1e-12, rtol=0)
end

# 3) P agit comme l'identité hors Span(S)
e = zeros(Float64, n); e[k+1] = 1.0
ye = zeros(Float64, n)
mul!(ye, P, e)
@test isapprox(ye, e; atol=1e-12, rtol=0)

# 4) SPD : v'Pv > 0 pour v ≠ 0
v = randn(n)
Pv = zeros(Float64, n)
mul!(Pv, P, v)
@test dot(v, Pv) > 0
end;

```

Test Summary:	Pass	Total	Time
Prec : opérateur de préconditionnement (Float64)	8	8	0.2s

Décomposition en valeur propres randomisée : intuition, calcul et usage

Objectif. Approcher les k plus grandes paires propres (λ_i, u_i) d'une matrice *symétrique* $A \in \mathbb{R}^{n \times n}$ en n'utilisant que des produits matrice-vecteur, sans factorisation complète.

Intuition.

On échantillonne le sous-espace propre dominant via un *sketch* aléatoire, puis on applique une projection de Rayleigh–Ritz à petite dimension.

Algorithme (schéma utilisé).

Soient k le rang cible, p le sur-échantillonnage et $\ell = k + p$.

1. Tirer $\Omega \in \mathbb{R}^{n \times \ell}$ (gaussien standard), puis former

$$Y = A\Omega.$$

2. Orthonormaliser Y par QR pour obtenir $Q \in \mathbb{R}^{n \times \ell}$.
3. (Optionnel) **Itérations de puissance** q fois :

$$Y \leftarrow AQ, \quad QR \Rightarrow Q.$$

But : améliorer la séparation spectrale quand le spectre décroît lentement.

4. Construire la petite matrice de Rayleigh

$$T = Q^\top A Q \in \mathbb{R}^{\ell \times \ell}, \quad TW = W\Lambda.$$

5. **Paires de Ritz** (approx. des paires propres de A) :

$$U_k = Q W_{(:, \ell-k+1:\ell)}, \quad \lambda_k = \text{diag}(\Lambda_{(\ell-k+1:\ell)}).$$

6. **Certification** (résidus) :

$$r_i = \|Au_i - \lambda_i u_i\|_2, \quad i = 1, \dots, k.$$

Usage en assimilation.

Utilisation des paires dominants retourner par l'algorithme pour construire le préconditionneur spectrale comme déjà expliquer.

Test de la fonction revd

```
@testset "revd : eigenpairs & puissance (q) améliorent les résidus" begin
    using Random, LinearAlgebra, Test
    Random.seed!(1234)

    # Matrice symétrique connue : A = Q*Diag()*Q'
    n = 50
    k = 5
    p = 45          # échantillonner tout l'espace, les k-dominants
                    #doivent etre les memes que ceux calculer avec algorithme exacte

    q0 = 0
    q2 = 5
    = 0.82

    # spectre décroissant bien séparé
    _true = _true = [100.0 * ^{(i-1)} for i in 1:n] # 100, ..., 1
    Q, _ = qr(randn(n,n))
    Q = Matrix(Q)
    A = Symmetric(Q * Diagonal(_true) * Q')

    # 1) Qualité des paires propres approx
    U0, k0, 0, res0 = revd(A, k; p=p, q=q0)
    # Les plus grands k exacts
    _top = _true[1:k]
    # Comparaison en valeur
    @test isapprox(sort(k0; rev=true), _top; rtol=1e-2, atol=1e-8)
```

```

# Résidus ||A*u - *u|| doivent être petits
@test maximum(res0) 1e-3 * norm(A)

# 2) Les itérations de puissance doivent améliorer (ou égaler) les résidus
p = 10 # l = k+p < n, on itère pour améliorer la qualité de l'approximation
      #vu qu'on échantillonne qu'une partie de l'image de A
U2, k2, 2, res2 = revd(A, k; p=p, q=q2)
@test maximum(res2) maximum(_top)*1e-6 + 1e-8

# 3) Dimensions / cohérence
@test size(U0) == (n, k)
@test length(k0) == k
@test length(res0) == k
@test 0 == minimum(k0)
# Orthonormalité (à tolérance num.)
@test isapprox(U0' * U0, I(k); atol=1e-8)
end;

```

Test Summary:	Pass	Total	Time
revd : eigenpairs & puissance (q) améliorent les résidus	8	8	0.7s

Expérience avec $n = 1000$ et $m = 400$

Initialisation du modèle et de l'opérateur d'observation

```

n = 100
nt = 6
dt = 0.025
F = 8.0
sigmaR = 0.1
total_space_obs = 20
total_time_obs = 4 # m = total_space_obs*total_time_obs
sigmaB = 0.8

# Model
model = Lorenz95Model(F, dt)

# Observations

space_inds_obs = round.(Int, range(1, n; length=total_space_obs))
time_inds_obs = round.(Int, range(1, nt-1; length=total_time_obs))
m = total_space_obs*total_time_obs
obs = ObsOperator(sigmaR, space_inds_obs, n, time_inds_obs, nt, m, model)
R = RMatrix(sigmaR)

# Background
B = BMatrix(sigmaB, n)

# Spin-up
xt = 3.0 .* ones(n) .+ randn(n)
xt = traj(model, xt, 5000)

# Background state
xb = xt .+ randn(n) .* sigmaB

```

```
# Observations
y = generate_obs(obs, xt);
```

Expérience sans préconditionnement des équations normales

```
iterates_GN = fourdvar(model, obs, R, B, xt, xb, y;
max_outer, prec = false, verbose=2);
```

Statistiques GN (partie 1)

	lsqr	lsmr	minres_qlp	minres
Outer-Iter	20.0	20.0	20.0	20.0
Inners-iter	710.0	704.0	908.0	905.0
grad	0.0128396	0.0128395	0.0128396	0.0128396
f(x_k)	80.7329	80.7329	80.7329	80.7329
err	0.17468	0.17468	0.17468	0.17468
time	0.073988	0.056092	0.0691635	0.0671056

Statistiques GN (partie 2)

cg_lanczos	cg	minares	diom	dqgmres
20.0	20.0	20.0	20.0	20.0
924.0	912.0	815.0	792.0	882.0
0.0128396	0.0128396	0.0127921	0.0128396	0.0128396
80.7329	80.7329	80.7329	80.7329	80.7329
0.17468	0.17468	0.17468	0.17468	0.17468
0.0676168	0.065261	0.0587026	0.0572155	0.0640482

Expérience avec préconditionnement des équations normales

```
iterates_GN = fourdvar(model, obs, R, B, xt, xb, y;
max_outer, prec = true, k = 10, p=10, = 1, q=2,verbose=2);
```

statistiques sur GN en utilisons différents sous-solveurs

	minres_qlp	minres	cg_lanczos	cg
Outer-Iter	20.0	20.0	20.0	20.0
Inners-iter	871.0	870.0	884.0	871.0
grad	0.0128396	0.0128396	0.0128396	0.0128396
f(x_k)	80.7329	80.7329	80.7329	80.7329
err	0.17468	0.17468	0.17468	0.17468
time	0.0875969	0.105871	0.119433	0.0843511

Analyse des résultats

Sans préconditionnement.

Nous observons que les méthodes `lsqr` et `lsmr` convergent plus rapidement que les algorithmes appliqués aux équations normales, et que `lsmr` est légèrement meilleur que `lsqr`. On remarque que les quantités

$$\|\text{err}\|, \quad |f(x_k)|$$

sont identiques à chiffres significatifs près pour les différents solveurs, mais la norme du gradient à la solution diffère légèrement d'un algorithme à l'autre.

Avec préconditionnement.

Premièrement, le temps de construction du préconditionneur n'est pas pris en compte : nous n'affichons que le temps total des sous-solveurs. Cela peut être justifié par le fait que le processus d'estimation peut être effectué en parallèle.

Le paramètre $\ell = 1$ signifie qu'à chaque itération externe, on construit le préconditionneur ; $k = 100$ signifie que le préconditionneur est construit avec les k paires dominantes des équations normales ; enfin, $q = 2$ indique que le passage de A est effectuée deux fois.

On remarque que le nombre d'itérations diminue pour les différents solveurs, ainsi que le temps d'exécution. Néanmoins, on n'arrive pas à converger plus rapidement que `lsqr` et `lsmr`. Si les opérations liées au préconditionneur étaient effectuées en parallèle, on pourrait bien sûr gagner en temps.

Les inconvénients d'une telle approche sont que, k est relativement grande dans ce cas de figure (distribution de spectre pas très favorable), pour obtenir une bonne approximation, il faut passer plusieurs fois par la matrice, et pour que le préconditionneur soit efficace, il faut une décroissance rapide du spectre (dans ce cas, une seule passe sur la matrice suffit — ce qui est un bon point).

Difficulté rencontrés

Vu que les équations normales s'écrivent sous la forme $X^\top X$, nous avons d'abord pensé à approximer la SVD de X en utilisant un algorithme de sketching, puis à en déduire une approximation de la décomposition en valeurs propres (EVD) de $X^\top X$. Cette méthode ne fonctionnait pas bien, nous avons donc envisagé d'appliquer directement l'EVD sur $X^\top X$, ce qui a mieux marché.

Objectifs accompli

- Implémentation d'un modèle non linéaire 'Lorenz95' au lieu d'un modèle linéaire 'ShallowWater'.
- Utilisation de différents algorithmes de 'Krylov' pour la résolution des sous-problèmes d'assimilation de données.
- Préconditionnement des équations normales en utilisant des algorithmes aléatoires.

Impact des résultats

Ce projet montre bien que les méthodes de moindres carrés linéaires sont plus performantes en temps que les méthodes basées sur les équations normales dans ce cas de figure. (si les critères d'arrêt ne se limitent pas simplement au nombre d'itérations!!)

Perspective personnelle

- Le calcul en parallèle (via l'utilisation de GPU) peut rendre l'utilisation des préconditionneurs spectraux intéressante.
- L'utilisation de matrices diagonales comme matrices de covariance n'est pas réaliste et n'influence pas suffisamment la décroissance du spectre résultant. La question se pose alors de savoir si d'autres manières de modéliser ces matrices pourraient induire une décroissance rapide du spectre, ce qui rendrait l'utilisation du préconditionneur `revd` pertinente. On pourrait chercher à modéliser une telle matrice en gardant à l'esprit l'objectif d'obtenir une forte décroissance spectrale.
- Comprendre la relation entre l'approximation de la SVD de X et l'approximation de l'EVD de $X^\top X$.
- Rendre le code actuel plus générique et optimiser.

Lien Github

- <https://github.com/Oussamamouhtal/DataAssim.jl>
- Le modèle Lorenze95 est implémenté dans “src/lorenz95.jl” (Annexe)
- Les opérateurs nécessaires sont implémentées dans “src/operators.jl” (Annexe)

Références

- N. Halko, P.-G. Martinsson, J. A. Tropp (2011). *Finding Structure with Randomness : Probabilistic Algorithms for Constructing Approximate Matrix Decompositions*. SIAM Review, 53(2), 217–288.

Annexe

```
"""
Structure représentant l'opérateur d'observation pour 4D-Var.

# Champs
- `sigmaR` : écart-type du bruit d'observation (diagonale de R)
- `space_inds` : indices des variables d'état observées (spatialement)
- `n` : taille de l'état total
- `time_inds` : indices temporels des observations (pour 4D-Var)
- `nt` : nombre total de pas de temps dans la trajectoire
- `model` : le modèle de dynamique (ex. Lorenz95Model)
"""

struct ObsOperator
    sigmaR::Float64
    space_inds::Vector{Int}
    n::Int
    time_inds::Vector{Int}
    nt::Int
    m::Int
    model::Lorenz95Model
end

"""
Génère des observations bruitées à partir d'un état de vérité xt.
"""

function generate_obs(op::ObsOperator, xt::Vector{Float64})
    x = copy(xt)
    y = Vector{Vector{Float64}}{ }()
    if op.nt > 0
        # 4DVAR
        counter = 0
        for ii in 0:op.nt-1
            if ii in op.time_inds
                new_obs = hop(op, x) .+ op.sigmaR .* randn(length(op.space_inds))
                push!(y, new_obs)
                counter += 1
            end
            counter == length(op.time_inds) && break
            x = traj(op.model, x, 1)
        end
    end
end
```

```

        return reduce(vcat, y)
    else
        # 3DVAR
        return hop(op, x) .+ op.sigmaR .* randn(length(op.space_inds))
    end
end

"""" Extraction de l'observation (opérateur H appliqué à x). """
function hop(op::ObsOperator, x::Vector{Float64})
    return x[op.space_inds]
end

"""" Modèle tangent de l'opérateur d'observation (H appliqué à dx). """
function tlm_hop(op::ObsOperator, dx::Vector{Float64})
    return dx[op.space_inds]
end

"""" Modèle adjoint de l'opérateur d'observation (H^T appliqué à ay). """

function adj_hop(op::ObsOperator, ay::Vector{Float64})
    ax = zeros(op.n)
    ax[op.space_inds] .= ay
    return ax
end
""""
Calcule le résidu d'observation : y - H(x), en 3D ou 4D.
""""
function misfit(op::ObsOperator, y::Vector{Float64}, xt::Vector{Float64})
    x = copy(xt)
    l = length(op.time_inds)
    nx = length(op.space_inds)
    if l == 0
        return y .- hop(op, x)
    else
        yy = reshape(copy(y), (nx, l))
        d = Vector{Vector{Float64}}{ }()
        counter = 0
        for ii in 0:op.nt-1
            if ii in op.time_inds
                push!(d, yy[:, counter+1] .- hop(op, x))
                counter += 1
            end
            counter == l && break
            x = traj(op.model, x, 1)
        end
        return reduce(vcat, d)
    end
end

"""" Applique l'opérateur G(x) = H o M(x) avec M model dynamique (traj)""""

function gop(op::ObsOperator, xt::Vector{Float64})
    x = copy(xt)

```



```

gx = Vector{Vector{Float64}}()
counter = 0
for ii in 0:op.nt-1
    if ii in op.time_inds
        push!(gx, hop(op, x))
        counter += 1
    end
    counter == length(op.time_inds) && break
    x = traj(op.model, x, 1)
end
return reduce(vcat, gx)
end

function tlm_gop(op::ObsOperator, xt::Vector{Float64}, dxt::Vector{Float64})
    x = copy(xt)
    dx = copy(dxt)
    dgx = Vector{Vector{Float64}}()
    counter = 0

    for ii in 0:op.nt-1
        if ii in op.time_inds
            push!(dgx, tlm_hop(op, dx))
            counter += 1
        end
        counter == length(op.time_inds) && break
        dx = tlm_traj(op.model, x, dx, 1)
        x = traj(op.model, x, 1)
    end

    return reduce(vcat, dgx)
end

function adj_gop(op::ObsOperator, xt::Vector{Float64}, axt::Vector{Float64})
    x = copy(xt)
    l = length(op.time_inds)
    nx = length(op.space_inds)
    agx = reshape(copy(axt), (nx, l))
    ax = [zeros(length(xt)) for _ in 1:(op.time_inds[end]+1)]
    counter = 0

    # Forward pass: stocke la trajectoire complète
    traj_xx = Vector{Vector{Float64}}()
    for _ in 1:op.nt
        push!(traj_xx, copy(x))
        x = traj(op.model, x, 1)
    end

    # Backward pass
    for ii in reverse(0:op.time_inds[end])
        if ii in reverse(op.time_inds)
            idx = findfirst(x -> x == ii, op.time_inds)
            ax[ii+1] .+= adj_hop(op, agx[:, idx])
            counter += 1
        end
    end
end

```

```

        end
        counter == l+1 && break
        if ii > 0
            ax[ii] = ad_traj(op.model, traj_xx[ii], ax[ii+1], 1)
        end
    end
    return ax[1]
end

# -----
# Matrices d'erreur de covariance d'observation et de background
# -----

""" Structure représentant R, la matrice de covariance des observations. """
struct RMatrix
    sigmaR::Float64
end

""" Applique R-1 à un vecteur. """
function invdot(R::RMatrix, d::Vector{Float64})
    return d ./ (R.sigmaR^2)
end

function sqrtinvdot(R::RMatrix, d::Vector{Float64})
    return d ./ (R.sigmaR)
end

""" Structure représentant la matrice de covariance B. """
mutable struct BMatrix
    sigmaB::Float64
    n::Int
end

function invdot(B::BMatrix, x::Vector{Float64})
    return x ./ (B.sigmaB^2)
end

function sqrtinvdot(B::BMatrix, x::Vector{Float64})
    return x ./ (B.sigmaB)
end

function Bdot(B::BMatrix, x::Vector{Float64})
    return x .* (B.sigmaB^2)
end

# Hessienne 4DVAR
struct Hessian4DVar
    obs::ObsOperator
    R::RMatrix
    B::BMatrix
end

```

```

    xt::Vector{Float64}
end

function LinearAlgebra.mul!(y, H::Hessian4DVar, dx::Vector{Float64})
    w = invdot(H.R, tlm_gop(H.obs, H.xt, dx))
    gtrinv_dx = adj_gop(H.obs, H.xt, w)
    binv_dx = invdot(H.B, dx)
    y .= binv_dx .+ gtrinv_dx
    return y
end

Base.size(H::Hessian4DVar) = (n, n)
Base.eltype(::Hessian4DVar) = Float64

# Jacobian 4DVAR
struct Jacobian4DVar
    obs::ObsOperator
    R::RMatrix
    B::BMatrix
    xt::Vector{Float64}
end

function LinearAlgebra.mul!(y, J::Jacobian4DVar, dx::Vector{Float64})
    v = sqrtinvdot(J.R, tlm_gop(J.obs, J.xt, dx))
    w = sqrtinvdot(J.B, dx)
    y .= [v; w]
    return y
end

Base.size(J::Jacobian4DVar) = (J.obs.n+J.obs.m, J.obs.n)
Base.eltype(J::Jacobian4DVar) = Float64

# Wrapper pour l'adjoint
struct Jacobian4DVarAdj
    J::Jacobian4DVar
end

LinearAlgebra.adjoint(J::Jacobian4DVar) = Jacobian4DVarAdj(J)

# Multiplication par l'adjoint : y ← J' * z
function LinearAlgebra.mul!(y::AbstractVector, JT::Jacobian4DVarAdj, z::AbstractVector)
    J = JT.J
    m, n = J.obs.m, J.obs.n
    @assert length(z) == m + n
    @assert length(y) == n

    z1 = z[1:m]      # partie "observation"
    z2 = z[m+1:end]  # partie "background"

    # Appliquer R^{-1/2} et B^{-1/2}
    rhalf_z1 = sqrtinvdot(J.R, z1)
    bhalf_z2 = sqrtinvdot(J.B, z2)

```

```

    t = adj_gop(J.obs, J.xt, rhalf_z1)

    # Somme des deux contributions
    y .= t .+ bhalf_z2
    return y
end

Base.size(JT::Jacobian4DVarAdj) = (JT.J.obs.n, JT.J.obs.m + JT.J.obs.n)
Base.eltype(JT::Jacobian4DVarAdj) = Float64

"""
    revd(A::AbstractMatrix, k; p=10, q=0)

Randomized EVD pour matrice symétrique A (n×n).
Retourne (U, , min, resnorms) avec les k plus grandes valeurs propres.
Toutes les multiplications par A sont réalisées en boucle via `mul!`.
"""
function revd(A, k::Integer; p::Integer=10, q::Integer=0)
    n, m = size(A)
    @assert n == m "A doit être carrée n×n"
    = k + p

    # 1) Matrice aléatoire  $\Omega$  (n× )
     $\Omega$  = randn(n, )

    # 2)  $Y = A * \Omega$ 
    Y = Matrix{eltype(A)}(undef, n, )
    Q = Matrix{eltype(A)}(undef, n, )
    tmp = similar( $\Omega$ , n) # vecteur de travail de longueur n
    for j in 1:
        mul!(tmp, A,  $\Omega[:,j]$ ) # tmp = A *  $\Omega[:,j]$ 
        copyto!(view(Y, :, j), tmp)
    end

    # 3) QR
    F = qr(Y)
    Q = Matrix(F.Q) # n×

    # 4) Itérations de puissance
    for _ in 1:q
        for j in 1:
            mul!(tmp, A, Q[:,j]) # tmp = A * Q[:,j]
            copyto!(view(Y, :, j), tmp)
        end
        F = qr(Y)
        Q = Matrix(F.Q)
    end

    # 5) AQ = A * Q
    AQ = Matrix{eltype(A)}(undef, n, )
    for j in 1:
        mul!(tmp, A, Q[:,j])

```

```

        copyto!(view(AQ, :, j), tmp)
    end

    # 6) Petite matrice de Rayleigh et EVD
    T = Symmetric(Q' * AQ)          # ×
    E = eigen(T)                    # valeurs triées ↑
    = E.values
    W = E.vectors
    idx = (-k+1):                   # indices des k plus grandes
    k = [idx]
    Uk = Q * W[:, idx]              # U   Q * W_k
    # 7) Résidus ||A*Uk[:,i] - k[i]*Uk[:,i]|| (en boucle)
    resnorms = similar(k)
    for i in 1:k
        mul!(tmp, A, Uk[:, i])      # tmp = A * u_i
        resnorms[i] = norm(tmp .- k[i] .* view(Uk, :, i))
    end

    return Uk, k, minimum(k), resnorms
end

struct Prec
    S::Matrix{Float64}
    Λ::Vector{Float64}
    ::Float64
end

"""
P = In + U(tetas-1/2 - I1)Ut
"""

function LinearAlgebra.mul!(y::AbstractVector, P::Prec, x::AbstractVector)
    t = P.S' * x
    t .= (sqrt(P.)/sqrt(P.Λ) .- 1) .* t
    t = P.S * t
    y .= t .+ x
    return y
end

Base.size(P::Prec) = (size(P.S, 1), size(P.S, 1))
Base.eltypes(P::Prec) = Float64;

```

Warning: Replacing docs for `Main.Notebook.ObsOperator :: Union{}` in module `Main.Notebook`

@ Base.Docs docs/Docs.jl:243

Warning: Replacing docs for `Main.Notebook.generate\_obs :: Tuple{Main.Notebook.ObsOperator, Vector{Float64}}` in module `Main.Notebook`

@ Base.Docs docs/Docs.jl:243

Warning: Replacing docs for `Main.Notebook.hop :: Tuple{Main.Notebook.ObsOperator, Vector{Float64}}` in module `Main.Notebook`

@ Base.Docs docs/Docs.jl:243

Warning: Replacing docs for `Main.Notebook.tlm\_hop :: Tuple{Main.Notebook.ObsOperator, Vector{Float64}}` in module `Main.Notebook`

@ Base.Docs docs/Docs.jl:243

Warning: Replacing docs for `Main.Notebook.misfit :: Tuple{Main.Notebook.ObsOperator, Vector{Float64}}` in module `Main.Notebook`

@ Base.Docs docs/Docs.jl:243

```

Warning: Replacing docs for `Main.Notebook.RMatrix :: Union{}` in module `Main.Notebook`
@ Base.Docs docs/Docs.jl:243
Warning: Replacing docs for `Main.Notebook.invdot :: Tuple{Main.Notebook.RMatrix, Vector{Float64}}` in
@ Base.Docs docs/Docs.jl:243
Warning: Replacing docs for `Main.Notebook.revd :: Tuple{Any, Integer}` in module `Main.Notebook`
@ Base.Docs docs/Docs.jl:243
Warning: Replacing docs for `LinearAlgebra.mul! :: Tuple{AbstractVector, Main.Notebook.Prec, AbstractV
@ Base.Docs docs/Docs.jl:243

```

```

using LinearAlgebra

export Lorenz95Model

struct Lorenz95Model
    F::Float64
    dt::Float64
end

function l95(model::Lorenz95Model, xx::Vector{Float64})
    n = length(xx)
    dxdt = zeros(Float64, n)
    for i in 1:n
        im2 = i - 2
        im1 = i - 1
        ip1 = i + 1

        if im2 < 1
            im2 += n
        end
        if im1 < 1
            im1 += n
        end
        if ip1 > n
            ip1 -= n
        end
        dxdt[i] = (xx[ip1] - xx[im2]) * xx[im1] - xx[i] + model.F
    end
    return dxdt
end

function tlm_l95(model::Lorenz95Model, x::Vector{Float64}, dx::Vector{Float64})
    n = length(x)
    ddxdt = zeros(Float64, n)

    for i in 1:n
        im2 = i - 2
        im1 = i - 1
        ip1 = i + 1

        if im2 < 1
            im2 += n
        end
        if im1 < 1

```

```

        im1 += n
    end
    if ip1 > n
        ip1 -= n
    end

    ddxdt[i] = (dx[ip1] - dx[im2]) * x[im1] + (x[ip1] - x[im2]) * dx[im1] - dx[i]
end

return ddxdt
end

function ad_l95(model::Lorenz95Model, x::Vector{Float64}, ax::Vector{Float64})
    n = length(x)
    adxdt = zeros(Float64, n)

    for i in 1:n
        im2 = i - 2
        im1 = i - 1
        ip1 = i + 1

        if im2 < 1
            im2 += n
        end
        if im1 < 1
            im1 += n
        end
        if ip1 > n
            ip1 -= n
        end

        adxdt[im2] -= x[im1] * ax[i]
        adxdt[im1] += x[ip1] * ax[i]
        adxdt[im1] -= x[im2] * ax[i]
        adxdt[ip1] += x[im1] * ax[i]
        adxdt[i]    -= ax[i]
    end

    return adxdt
end

function RKstep(model::Lorenz95Model, xx::Vector{Float64})
    dt = model.dt
    k1 = l95(model, xx)
    k2 = l95(model, xx .+ (dt/2) .* k1)
    k3 = l95(model, xx .+ (dt/2) .* k2)
    k4 = l95(model, xx .+ dt .* k3)
    return xx .+ (dt/6) .* (k1 .+ 2*k2 .+ 2*k3 .+ k4)
end

function dRKstep(model::Lorenz95Model, xx::Vector{Float64}, dx::Vector{Float64})

```

```

dt = model.dt
k1 = 195(model, xx)
dk1 = tlm_195(model, xx, dx)
k2 = 195(model, xx .+ (dt/2) .* k1)
dk2 = tlm_195(model, xx .+ (dt/2) .* k1, dx .+ (dt/2) .* dk1)
k3 = 195(model, xx .+ (dt/2) .* k2)
dk3 = tlm_195(model, xx .+ (dt/2) .* k2, dx .+ (dt/2) .* dk2)
dk4 = tlm_195(model, xx .+ dt .* k3, dx .+ dt .* dk3)
return dx .+ (dt/6) .* (dk1 .+ 2*dk2 .+ 2*dk3 .+ dk4)
end

function arKstep(model::Lorenz95Model, xx::Vector{Float64}, axp::Vector{Float64})
dt = model.dt
x0 = copy(xx)
k1 = 195(model, x0)
x1 = x0 .+ (dt/2) .* k1
k2 = 195(model, x1)
x2 = x0 .+ (dt/2) .* k2
k3 = 195(model, x2)
x3 = x0 .+ dt .* k3

ak1 = zeros(length(xx))
ak2 = zeros(length(xx))
ak3 = zeros(length(xx))
ak4 = zeros(length(xx))
ax = zeros(length(xx))

jak2 = zeros(length(xx))
jak3 = zeros(length(xx))
jak4 = zeros(length(xx))

ak4 .+= (dt/6) .* axp
ak3 .+= (dt/3) .* axp
ak2 .+= (dt/3) .* axp
ak1 .+= (dt/6) .* axp
ax .+= axp

jak4 .= ad_195(model, x3, ak4)
ak3 .+= dt .* jak4
ax .+= jak4

jak3 .= ad_195(model, x2, ak3)
ak2 .+= (dt/2) .* jak3
ax .+= jak3

jak2 .= ad_195(model, x1, ak2)
ak1 .+= (dt/2) .* jak2
ax .+= jak2

ax .+= ad_195(model, x0, ak1)

return ax
end

```



```

function traj(model::Lorenz95Model, x::Vector{Float64}, nt::Int)
    xx = copy(x)
    for _ in 1:abs(nt)
        xx = RKstep(model, xx)
    end
    return xx
end

function tlm_traj(model::Lorenz95Model, x::Vector{Float64}, dx::Vector{Float64}, nt::Int)
    xx = copy(x)
    dxx = copy(dx)
    for _ in 1:abs(nt)
        dxx = dRKstep(model, xx, dxx)
        xx = RKstep(model, xx)
    end
    return dxx
end

function ad_traj(model::Lorenz95Model, x::Vector{Float64}, ax::Vector{Float64}, nt::Int)
    axx = copy(ax)
    xx = copy(x)
    traj_xx = Vector{Vector{Float64}}()
    for _ in 1:abs(nt)
        push!(traj_xx, xx)
        xx = RKstep(model, xx)
    end
    for i in 1:abs(nt)
        axx = aRKstep(model, traj_xx[nt - i + 1], axx)
    end
    return axx
end;

```